

Name: Rai Muhammad Umer

Sap Id:70154216

Section: 5-B

Project Assignment

**Github REPOSITORY LINK:**

<https://github.com/70154216-blip/Lab-Project>

## **LIBRARY MANAGEMENT SYSTEM**

### **TASK 1**

#### **Description:**

The Library Management System is used to manage books, students, and borrowing records in a library. The system helps librarians store and retrieve information efficiently. Few library management sqls and ho they work I provided few screenshots and github repository include .sql files

#### **Features:**

**Book Management** – Add, update, delete, and search books in the library database.

**Member Management** – Register and manage library members' information.

**Book Issue and Return** – Record borrowing and returning of books.

**Fine Management** – Calculate and collect fines for overdue books.

**Search Facility** – Search books by title, author, ISBN, or category.

**Inventory Management** – Track available, issued, and lost books.

**Reservation System** – Allow members to reserve books that are currently unavailable.

**Report Generation** – Generate reports on issued books, overdue books, and member activities.

**User Authentication** – Secure login for librarians and members.

**Notifications and Reminders** – Send alerts for due dates, overdue books, and reservations.

**Expected outcome:**

The system will maintain library records digitally and reduce manual paperwork. Count books, loan, ddl dml etc

**Members Table**

| Attribute | Data Type    | Description     |
|-----------|--------------|-----------------|
| member_id | INT          | Primary Key     |
| name      | VARCHAR(100) | Member Name     |
| email     | VARCHAR(100) | Email Address   |
| phone     | VARCHAR(20)  | Phone Number    |
| address   | VARCHAR(255) | Address         |
| join_date | DATE         | Membership Date |

**AUTHOR TABLE:**

| Attribute   | Data Type    | Description |
|-------------|--------------|-------------|
| author_id   | INT          | Primary Key |
| author_name | VARCHAR(100) | Author Name |

**Categories Table**

| Attribute     | Data Type    | Description   |
|---------------|--------------|---------------|
| category_id   | INT          | Primary Key   |
| category_name | VARCHAR(100) | Book Category |

**Books Table:**

| Attribute        | Data Type    | Description      |
|------------------|--------------|------------------|
| book_id          | INT          | Primary Key      |
| title            | VARCHAR(200) | Book Title       |
| isbn             | VARCHAR(20)  | ISBN Number      |
| author_id        | INT          | Foreign Key      |
| category_id      | INT          | Foreign Key      |
| total_copies     | INT          | Total Copies     |
| available_copies | INT          | Available Copies |

### Librarians Table:

| Attribute    | Data Type   | Description    |
|--------------|-------------|----------------|
| librarian_id | INT         | Primary Key    |
| username     | VARCHAR(50) | Login Username |

### Issued\_Books Table

| Attribute   | Data Type | Description        |
|-------------|-----------|--------------------|
| issue_id    | INT       | Primary Key        |
| book_id     | INT       | Foreign Key        |
| member_id   | INT       | Foreign Key        |
| issue_date  | DATE      | Book Issue Date    |
| due_date    | DATE      | Return Due Date    |
| return_date | DATE      | Actual Return Date |

---

### 7. Fines Table

| Attribute   | Data Type     | Description        |
|-------------|---------------|--------------------|
| fine_id     | INT           | Primary Key        |
| issue_id    | INT           | Foreign Key        |
| fine_amount | DECIMAL(10,2) | Fine Amount        |
| paid_status | VARCHAR(20)   | Paid/Unpaid Status |

---

### 8. Reservations Table

| Attribute        | Data Type   | Description                |
|------------------|-------------|----------------------------|
| reservation_id   | INT         | Primary Key                |
| book_id          | INT         | Foreign Key                |
| member_id        | INT         | Foreign Key                |
| reservation_date | DATE        | Reservation Date           |
| status           | VARCHAR(20) | Pending/Approved/Cancelled |

---

### 9. Notifications Table

| Attribute         | Data Type    | Description          |
|-------------------|--------------|----------------------|
| notification_id   | INT          | Primary Key          |
| member_id         | INT          | Foreign Key          |
| message           | VARCHAR(255) | Notification Message |
| notification_date | DATE         |                      |

### **Identify Relationships**

| Relationship           | Type         |
|------------------------|--------------|
| Student borrows Book   | Many-to-Many |
| Librarian manages Book | One-to-Many  |

### **Primary Keys (PK)**

- member\_id
- author\_id

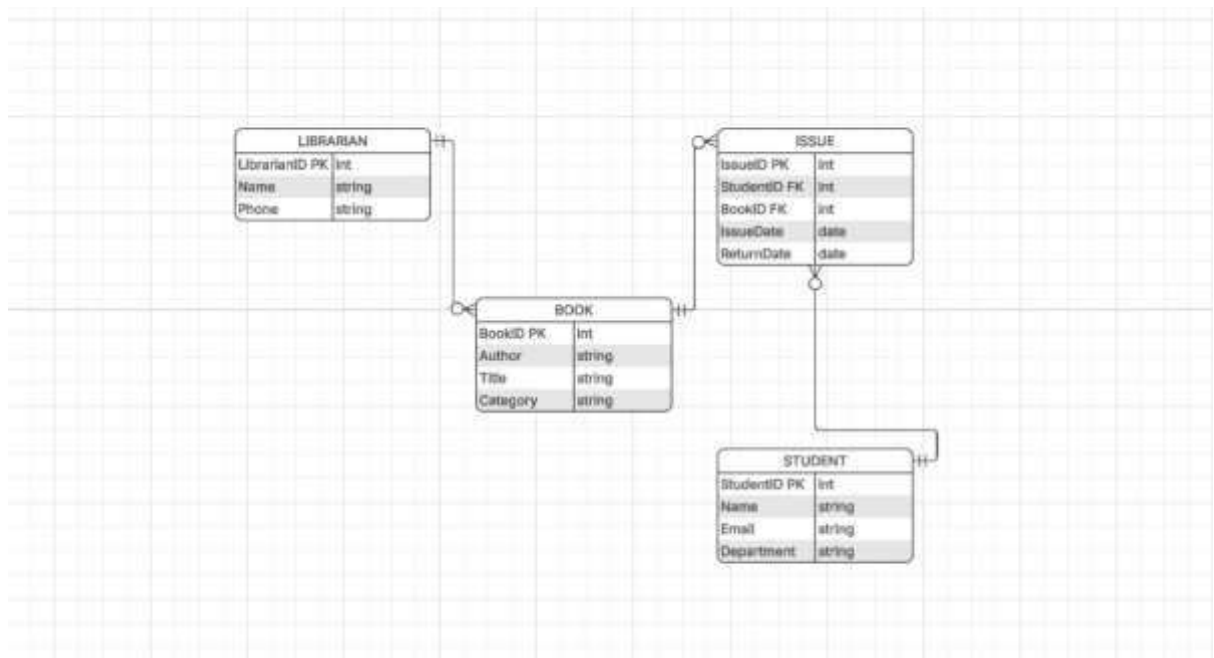
- category\_id
- book\_id
- librarian\_id
- issue\_id
- fine\_id
- reservation\_id
- notification\_id

### Foreign Keys (FK)

- Books.author\_id → Authors.author\_id
- Books.category\_id → Categories.category\_id
- Issued\_Books.book\_id → Books.book\_id
- Issued\_Books.member\_id → Members.member\_id
- Fines.issue\_id → Issued\_Books.issue\_id
- Reservations.book\_id → Books.book\_id
- Reservations.member\_id → Members.member\_id
- Notifications.member\_id → Members.member\_id

## TASK 3

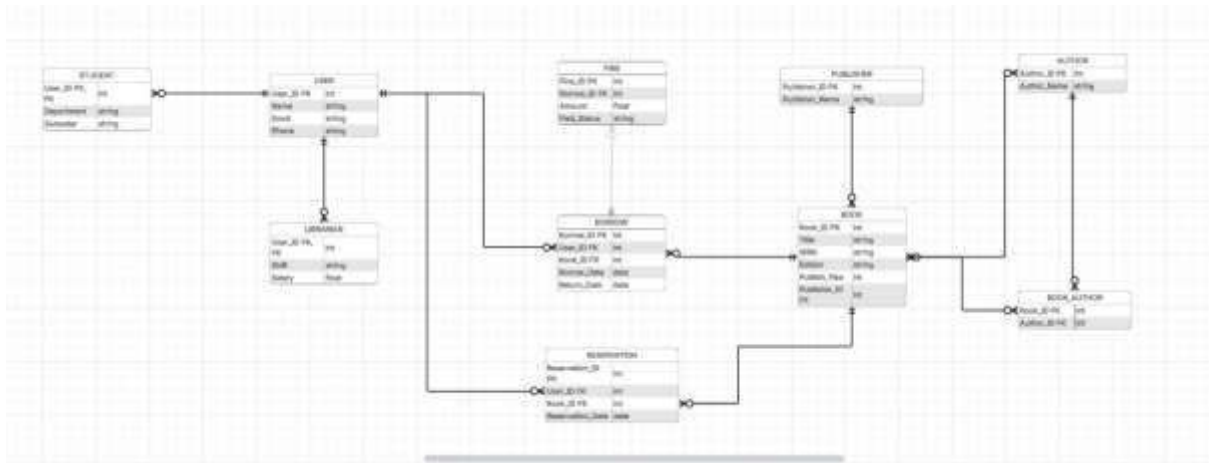
### ERD Diagram



Note: Created it on Lucid.

## EERD Diagram

By adding specialization/generalization we convert it into EERD.



Adding of new entities for EERD Diagram to make it look more clear.

### USER Entity

- User as a super class
- Student IS A User
- Librarian IS A User

This is proper specialization/generalization

### BORROW Entity

It correctly stores:

- which user borrowed
- which book
- borrow date
- return date

### FINE Entity

FINE → linked to BORROW

- Fine depends on borrowing transaction.

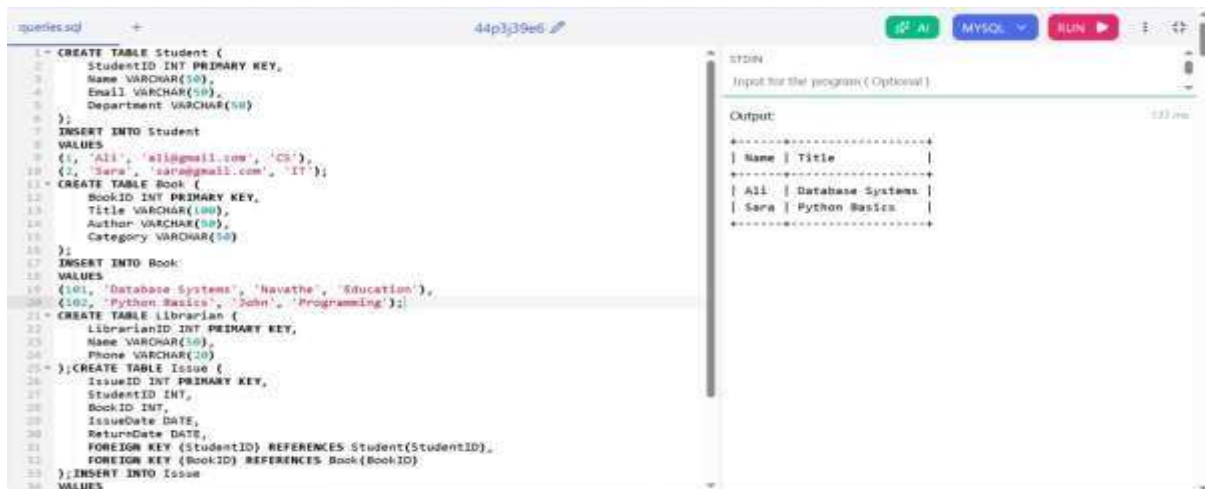
## RESERVATION Entity

Allows:

- users reserving books before borrowing.

## TASK 6

### CREATE TABLES:



The screenshot shows a MySQL IDE window with a SQL editor on the left and a console on the right. The SQL editor contains the following queries:

```
1 - CREATE TABLE Student (  
2   StudentID INT PRIMARY KEY,  
3   Name VARCHAR(50),  
4   Email VARCHAR(50),  
5   Department VARCHAR(50)  
6 );  
7 INSERT INTO Student  
8 VALUES  
9 ({}, 'Ali', 'ali@gmail.com', 'CS'),  
10 ({}, 'Sara', 'sara@gmail.com', 'IT');  
11 - CREATE TABLE Book (  
12   BookID INT PRIMARY KEY,  
13   Title VARCHAR(100),  
14   Author VARCHAR(50),  
15   Category VARCHAR(50)  
16 );  
17 INSERT INTO Book  
18 VALUES  
19 ({101, 'Database Systems', 'Navathe', 'Education'},  
20 {102, 'Python Basics', 'John', 'Programming'});  
21 - CREATE TABLE Librarian (  
22   LibrarianID INT PRIMARY KEY,  
23   Name VARCHAR(50),  
24   Phone VARCHAR(20)  
25 );  
26 - CREATE TABLE Issue (  
27   IssueID INT PRIMARY KEY,  
28   StudentID INT,  
29   BookID INT,  
30   IssueDate DATE,  
31   ReturnDate DATE,  
32   FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
33   FOREIGN KEY (BookID) REFERENCES Book(BookID)  
34 );  
35 INSERT INTO Issue  
36 VALUES
```

The console on the right shows the output of the queries, displaying a table with columns 'Name' and 'Title' and rows for 'Ali' and 'Sara'.

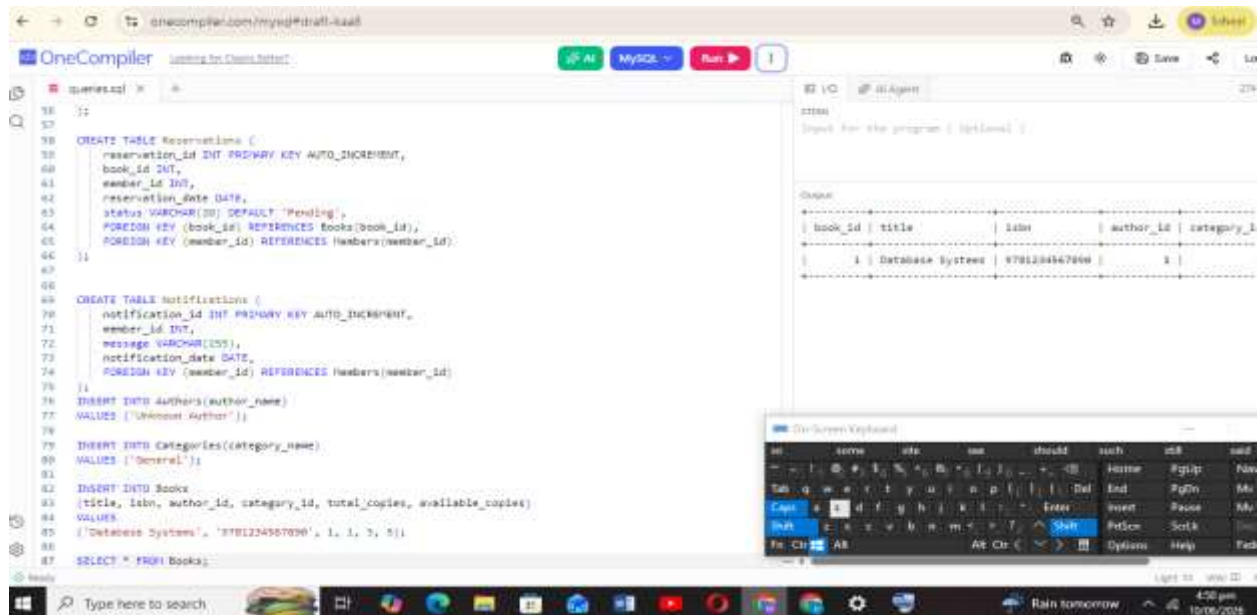
### ADD A BOOK:

```
INSERT INTO Authors(author_name)  
VALUES ('Unknown Author');
```

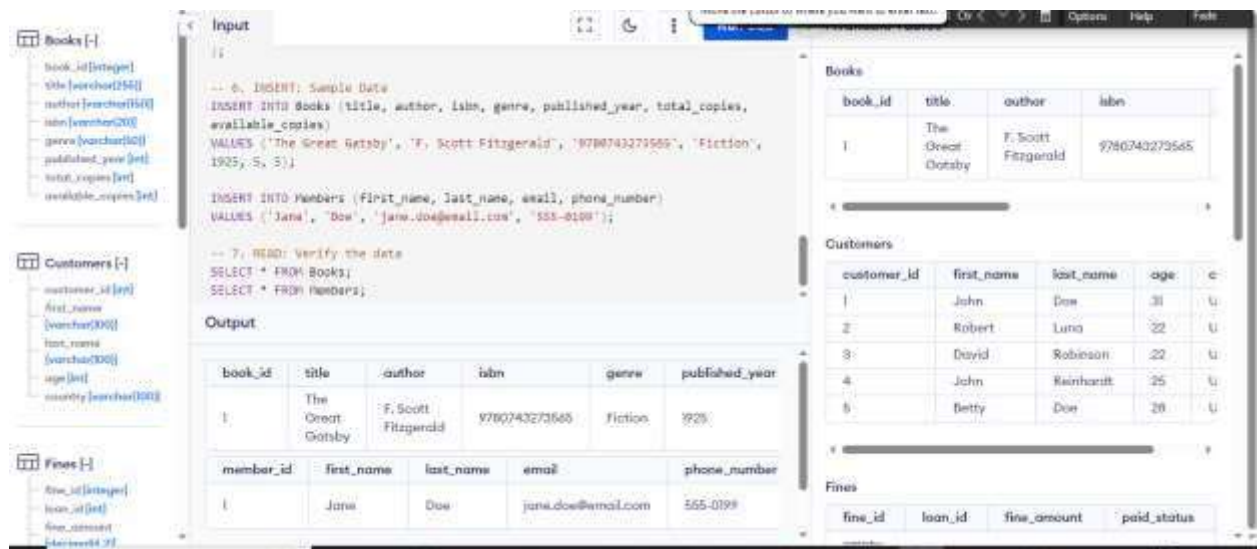
```
INSERT INTO Categories(category_name)  
VALUES ('General');
```

```
INSERT INTO Books  
(title, isbn, author_id, category_id, total_copies, available_copies)  
VALUES  
( 'Database Systems', '9781234567890', 1, 1, 5, 5);
```

```
SELECT * FROM Books;
```



Read a book:



Update books

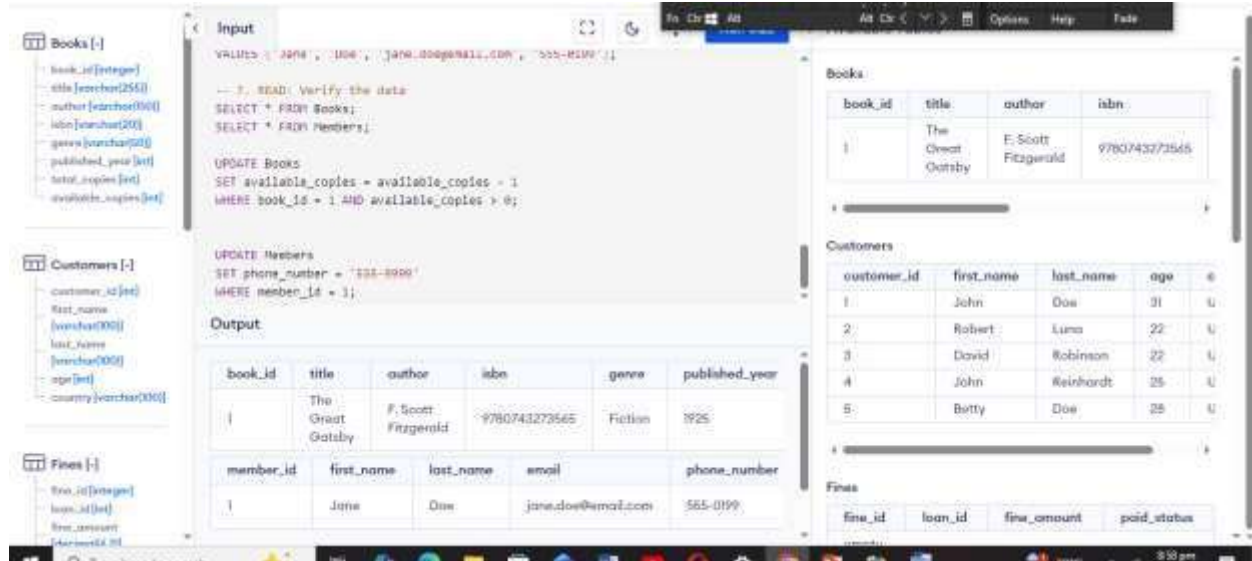
sql

UPDATE Books

```

SET available_copies = available_copies - 1
WHERE book_id = 1 AND available_copies > 0;
UPDATE Members
SET phone_number = '555-9999'
WHERE member_id = 1;

```



## Books

|  | published_year | total_copies | available_copies |
|--|----------------|--------------|------------------|
|  | 1925           | 5            | 4                |

## Loans:

### INNER JOIN:

```

JOIN Members ON Loans.member_id = Members.member_id JOIN
Books ON Loans.book_id = Books.book_id WHERE
Loans.return_date IS NULL; JOIN Members ON Loans.member_id =
Members.member_id JOIN Books ON Loans.book_id =
Books.book_id WHERE Loans.return_date IS NULL;

```

The screenshot shows the OneCompiler MySQL IDE. The left pane contains the following SQL queries:

```

32 -- 3. CREATE: Fines
33 CREATE TABLE Fines (
34   fine_id INT PRIMARY KEY AUTO_INCREMENT,
35   loan_id INT,
36   fine_amount DECIMAL(6,2),
37   paid_status VARCHAR(10) DEFAULT 'Unpaid',
38   FOREIGN KEY (loan_id) REFERENCES loans(loan_id)
39 );
40
41 SHOW TABLES;
42
43 DESCRIBE loans;
44
45
46
47
48
49
50
51
52 SELECT * FROM Books;
53
54 -- 4. INSERT DATA
55 INSERT INTO Books (title, author, isbn, total_copies, available_copies)
56 VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', '9780743273565', 5, 5);
57
58 INSERT INTO Members (first_name, last_name, email)
59 VALUES ('Jane', 'Doe', 'jane.doe@gmail.com');
60
61 -- 5. READ DATA (This will now work because due_date exists)
62 SELECT
63   a.first_name,
64   b.title,
65   l.due_date
66 FROM Loans l
67 JOIN Members m ON l.member_id = m.member_id
68 JOIN Books b ON l.book_id = b.book_id;

```

The right pane shows the output of the queries:

Output for the program (Optional):

```

Tables_in_sandbox_db
+-----+
| books |
| fines |
| loans |
| members |
+-----+

```

| Field       | Type | Null | Key | Default                       | Extra    |
|-------------|------|------|-----|-------------------------------|----------|
| loan_id     | int  | NO   | PK  | N/A                           | auto_inc |
| book_id     | int  | YES  | FK  | N/A                           |          |
| member_id   | int  | YES  | FK  | N/A                           |          |
| loan_date   | date | YES  |     | curdate()                     | DEFAULT  |
| due_date    | date | YES  |     | (curdate() + Interval 14 day) | DEFAULT  |
| return_date | date | YES  |     | N/A                           |          |

Count total books:

The screenshot shows the OneCompiler MySQL IDE. The left pane contains the following SQL queries:

```

30 CREATE TABLE Fines (
31   fine_id INT PRIMARY KEY AUTO_INCREMENT,
32   loan_id INT,
33   fine_amount DECIMAL(6,2),
34   paid_status VARCHAR(10) DEFAULT 'Unpaid',
35   FOREIGN KEY (loan_id) REFERENCES loans(loan_id)
36 );
37
38 SHOW TABLES;
39
40 DESCRIBE loans;
41
42
43
44
45
46
47
48
49
50
51
52 SELECT * FROM Books;
53
54 -- 4. INSERT DATA
55 INSERT INTO Books (title, author, isbn, total_copies, available_copies)
56 VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', '9780743273565', 5, 5);
57
58 INSERT INTO Members (first_name, last_name, email)
59 VALUES ('Jane', 'Doe', 'jane.doe@gmail.com');
60
61 -- 5. READ DATA (This will now work because due_date exists)
62 SELECT
63   a.first_name,
64   b.title,
65   l.due_date
66 FROM Loans l
67 JOIN Members m ON l.member_id = m.member_id
68 JOIN Books b ON l.book_id = b.book_id;
69
70 SELECT SUM(total_copies) AS grand_total_books FROM Books;

```

The right pane shows the output of the queries:

Output for the program (Optional):

```

Tables_in_sandbox_db
+-----+
| books |
| fines |
| loans |
| members |
+-----+

```

| Field       | Type | Null | Key | Default                       | Extra |
|-------------|------|------|-----|-------------------------------|-------|
| loan_id     | int  | NO   | PK  | N/A                           | auto  |
| book_id     | int  | YES  | FK  | N/A                           |       |
| member_id   | int  | YES  | FK  | N/A                           |       |
| loan_date   | date | YES  |     | curdate()                     | DEFA  |
| due_date    | date | YES  |     | (curdate() + Interval 14 day) | DEFA  |
| return_date | date | YES  |     | N/A                           |       |

Output for the program (Optional):

```

+-----+
| grand_total_books |
+-----+
| 5 |
+-----+

```

```

+-----+
+-----+
| grand_total_books |
+-----+
| 5 |
+-----+

```

## OVERDUEBOOKS:

```
SELECT * FROM Books;
SELECT m.name, b.title, i.due_date
FROM Issued_Books i
JOIN Members m ON i.member_id = m.member_id
JOIN Books b ON i.book_id = b.book_id
WHERE i.return_date IS NULL
AND i.due_date < CURDATE();
```

The screenshot shows the OneCompiler web IDE interface. The left pane displays a MySQL script with the following content:

```
66 CREATE TABLE notifications (
67   member_id INT,
68   message VARCHAR(100),
69   notification_date DATE,
70   FOREIGN KEY (member_id) REFERENCES Members(member_id)
71 );
72
73 INSERT INTO Authors (author_name)
74 VALUES ('Abraham Silberschatz');
75
76 INSERT INTO Categories (category_name)
77 VALUES ('Database');
78
79 INSERT INTO Members (name, email, phone, address, join_date)
80 VALUES ('Ali', 'ali@gmail.com', '99881234567', 'Istanbul', CURDATE());
81
82 INSERT INTO Books
83 (title, isbn, author_id, category_id, total_copies, available_copies)
84 VALUES
85 ('Database Systems', '9781234567890', 1, 1, 5, 5);
86
87 INSERT INTO Issued_Books
88 (book_id, member_id, issue_date, due_date, return_date)
89 VALUES
90 ((1, 1, CURDATE(), DATE_SUB(CURDATE(), INTERVAL 2 DAY), NULL));
91
92 SELECT * FROM Books;
93 SELECT m.name, b.title, i.due_date
94 FROM Issued_Books i
95 JOIN Members m ON i.member_id = m.member_id
96 JOIN Books b ON i.book_id = b.book_id
97 WHERE i.return_date IS NULL
98 AND i.due_date < CURDATE();
```

The right pane shows the output of the queries. The first query, 'SELECT \* FROM Books;', returns a single row:

| book_id | title            | isbn          | author_id | category_id | total_copies | available_copies |
|---------|------------------|---------------|-----------|-------------|--------------|------------------|
| 1       | Database Systems | 9781234567890 | 1         | 1           | 5            | 5                |

The second query, 'SELECT m.name, b.title, i.due\_date FROM Issued\_Books i JOIN Members m ON i.member\_id = m.member\_id JOIN Books b ON i.book\_id = b.book\_id WHERE i.return\_date IS NULL AND i.due\_date < CURDATE();', returns a single row:

| name | title            | due_date   |
|------|------------------|------------|
| Ali  | Database Systems | 2025-06-08 |

A Windows keyboard overlay is visible in the bottom right corner of the IDE window.

## FINES:

```
INSERT INTO Fines (issue_id, fine_amount)
VALUES (1, 100.00);
SELECT * FROM Fines;
```

```
1 INSERT INTO Authors (author_name)
2 VALUES ('Abraham Silberschatz');
3
4 INSERT INTO Categories (category_name)
5 VALUES ('Database');
6
7 INSERT INTO Members (name, email, phone, address, join_date)
8 VALUES ('Ali', 'ali@gmail.com', '03001134567', 'Lahore', CURDATE());
9
10 INSERT INTO Books
11 (title, isbn, author_id, category_id, total_copies, available_copies)
12 VALUES
13 ('Database Systems', '9781134567890', 1, 1, 5, 3);
14
15 INSERT INTO Issued_Books
16 (book_id, member_id, issue_date, due_date, return_date)
17 VALUES
18 (1, 1, CURDATE(), DATE_SUB(CURDATE(), INTERVAL 2 DAY), NULL);
19
20 SELECT * FROM Authors;
21 SELECT * FROM Categories;
22 SELECT * FROM Members;
23 SELECT * FROM Books;
24 SELECT * FROM Issued_Books;
25 SELECT * FROM Fines;
26
27 INSERT INTO Fines (issue_id, fine_amount)
28 VALUES (1, 100.00);
29 SELECT * FROM Fines;
```

Open for the program (Optional)

Output

| member_id | name | email         | phone       | address | join_date  |
|-----------|------|---------------|-------------|---------|------------|
| 1         | Ali  | ali@gmail.com | 03001134567 | Lahore  | 2026-06-10 |

| book_id | title            | isbn          | author_id | category_id |
|---------|------------------|---------------|-----------|-------------|
| 1       | Database Systems | 9781134567890 | 1         | 1           |

| issue_id | book_id | member_id | issue_date | due_date   | return_date |
|----------|---------|-----------|------------|------------|-------------|
| 1        | 1       | 1         | 2026-06-10 | 2026-06-08 | NULL        |

| fine_id | issue_id | fine_amount | paid_status |
|---------|----------|-------------|-------------|
| 1       | 1        | 100.00      | Unpaid      |

| fine_id | issue_id | fine_amount | paid_status |
|---------|----------|-------------|-------------|
| 1       | 1        | 100.00      | Unpaid      |

## Library Report:

```
SELECT b.title, COUNT(i.issue_id) AS times_issued
FROM Books b
LEFT JOIN Issued_Books i
ON b.book_id = i.book_id
GROUP BY b.book_id;
```

```

76 INSERT INTO Authors (author_name)
77 VALUES ('Abraham Silberschatz');
78
79 INSERT INTO Categories (category_name)
80 VALUES ('Database');
81
82 INSERT INTO Members (name, email, phone, address, join_date)
83 VALUES ('Ali', 'ali@gmail.com', '03001234567', 'Lahore', CURDATE());
84
85 INSERT INTO Books
86 (title, isbn, author_id, category_id, total_copies, available_copies)
87 VALUES
88 ('Database Systems', '9781234567890', 1, 1, 5, 3);
89
90 INSERT INTO Issued_Books
91 (book_id, member_id, issue_date, due_date, return_date)
92 VALUES
93 (1, 1, CURDATE(), DATE_SUB(CURDATE(), INTERVAL 2 DAY), NULL);
94
95 SELECT * FROM Authors;
96 SELECT * FROM Categories;
97 SELECT * FROM Members;
98 SELECT * FROM Books;
99 SELECT * FROM Issued_Books;
100 SELECT * FROM #lines;
101
102 SELECT b.title, COUNT(i.issue_id) AS times_issued
103 FROM Books b
104 LEFT JOIN Issued_Books i
105 ON b.book_id = i.book_id
106 GROUP BY b.book_id;
107

```

Input for the program (optional)

Output

| author_id | author_name          |
|-----------|----------------------|
| 1         | Abraham Silberschatz |

| category_id | category_name |
|-------------|---------------|
| 1           | Database      |

| member_id | name | email         | phone       | address | join_date  |
|-----------|------|---------------|-------------|---------|------------|
| 1         | Ali  | ali@gmail.com | 03001234567 | Lahore  | 2026-06-10 |

| book_id | title            | isbn          | author_id | category_id |
|---------|------------------|---------------|-----------|-------------|
| 1       | Database Systems | 9781234567890 | 1         | 1           |

| issue_id | book_id | member_id | issue_date | due_date | return_date |
|----------|---------|-----------|------------|----------|-------------|
|----------|---------|-----------|------------|----------|-------------|

Light 11 100 00 MySQL